

Meshr-Lang — Documentation du langage

Version : 0.1.0

Statut : En conception active

Mainteneur : G. Prince - Architecte Principal Dernière mise à jour : 2025-09-10

🔖 Objectif du langage

Meshr-Lang est un langage déclaratif dédié à la description des artefacts d'une architecture **Data-as-a-Product**, tels que les domaines, produits de données, contrats, aspects (métadonnées), équipes et politiques.

Il vise à offrir une syntaxe lisible, formelle et exécutable pour :

- Structurer la documentation vivante du data mesh
- Générer automatiquement les représentations techniques (YAML, Rego, Terraform, etc.)
- Alimenter les registres, catalogues et plateformes self-service

🔖 Syntaxe de base

- Fichiers : `.meshr`
- Style : déclaratif à blocs (inspiré de HCL, Kotlin DSL, GraphQL SDL)
- Commentaires : `// commentaire ligne` ou `/* commentaire bloc */`
- Valeurs prises en charge :
 - Chaînes : `"texte"`
 - Booléens : `true`, `false`
 - Nombres : décimaux (`42`, `3.14`) et hexadécimaux (`0xFF`, `0x00FF00`)
 - Noms qualifiés : `privacy.Level.HIGH`
 - Intervalles : `Interval 12 month`, `Interval -1 day`, `Interval "2024-01" year to month`

🔖 Commentaires

- Les commentaires sur une seule ligne commencent par `//`
- Les commentaires multi-lignes sont délimités par `/*` et `*/`
- Ils sont ignorés par l'analyse syntaxique (non représentés dans l'AST)

Exemples :

```
// Ceci est un commentaire simple

/*
Ceci est un commentaire
sur plusieurs lignes
*/
```

🔖 Déclaration des modules et imports

La première unité structurante de Meshr est le **module**. Il représente un espace de noms versionné, à partir duquel les autres artefacts sont déclarés ou importés.

🔖 Syntaxe

```

@experimental
@version("0.2.0")
module marketing.analytics

import Customer from marketing.shared           // ☑ forme simple
import { Customer } from marketing.shared       // ☑ forme équivalente
import { * } from core.types                   // ☑ importation complète
import * from core.types                       // ☑ forme équivalente

import Customer, Producer from marketing.shared // ☑ interdit : nécessite des accolades

// autres déclarations

```

☑ Sémantique

- Un fichier `.meshr` = un module unique.
- Le module définit le **namespace** des artefacts qu'il contient.
- Les `import` permettent d'accéder à des symboles publics d'autres modules.
- Des **annotations peuvent précéder** la déclaration module (ex.: `@experimental`, `@version("...")`).
- Les `import` permettent d'accéder à des symboles **explicitement exportés** d'autres modules.
- Les éléments d'un module **ne sont pas visibles depuis l'extérieur** s'ils ne sont pas précédés du mot-clé `export`.
- L'import de `{ * }` signifie **importer tous les symboles exportés** du module cible.
- L'import de `{ A, B }` permet une **importation sélective**.
- Les accolades `{ }` sont **optionnelles uniquement si un seul identifiant** est importé.
- Si plusieurs identifiants sont importés d'un même module, l'usage des accolades est **obligatoire**.

☑ Export des déclarations

Deux formes d'export sont possibles dans un module :

☑ Export inline (immédiat)

```
export enum SensitivityLevel is (public, internal, restricted)
```

- La déclaration est rendue publique immédiatement.
- Facile à lire mais répétitif si plusieurs artefacts sont à exporter.

☑ Export groupé (centralisé)

```
export { PIICategory, SensitivityLevel }

enum PIICategory is (contact, identity, financial, health, biometric, location)
enum SensitivityLevel is (public, internal, restricted, confidential)
```

- Doit apparaître juste **après les imports**, jamais en fin de module.
- Permet de déclarer tous les artefacts exportés en un seul point.
- Aucune déclaration listée dans ce bloc n'a besoin du mot-clé `export` en ligne.

☑ Règles de priorité d'export

1. Une déclaration précédée de `export` est **immédiatement publique**.
2. Une déclaration listée dans `export { ... }` devient **publique à posteriori**.
3. Un même nom peut apparaître dans les deux formes (toléré).
4. Toute déclaration **non exportée explicitement** est **privée** au module.

- Un fichier `.meshr` = un module unique.
- Le module définit le **namespace** des artefacts qu'il contient.
- Les `import` permettent d'accéder à des symboles publics d'autres modules.
- Les annotations `@module_*` sont optionnelles mais encouragées pour documenter les modules.
- Les `import` permettent d'accéder à des symboles **explicitement exportés** d'autres modules.
- Les éléments d'un module **ne sont pas visibles depuis l'extérieur** s'ils ne sont pas précédés du mot-clé `export`.
- L'import de `{ * }` signifie **importer tous les symboles exportés** du module cible.
- L'import de `{ A, B }` permet une **importation sélective**.
- Les accolades `{ }` sont **optionnelles uniquement si un seul identifiant** est importé.
- Si plusieurs identifiants sont importés d'un même module, l'usage des accolades est **obligatoire**.

☑ Convention d'arborescence

Par convention (non obligatoire), l'arborescence des fichiers reflète les noms qualifiés des modules :

Module	Fichier
core.types	modules/core/types.meshr
marketing.analytics	modules/marketing/analytics.meshr

Annotations

Les **annotations** permettent d'ajouter des métadonnées structurées sur n'importe quelle déclaration du langage Meshr (`module`, `product`, `domain`, `enum`, etc.). Elles sont inspirées de langages comme Java, Kotlin ou GraphQL.

Syntaxe

- Une annotation commence par `@` suivie de son nom.
- Elle peut recevoir :
 - Aucun argument : `@experimental`
 - Une seule valeur : `@since("1.2.0")`
 - Une ou plusieurs paires clé-valeur : `@author(name="Greg")`
 - Des arguments mixtes : `@scope(level=ScopeLevel.Internal)`
- Plusieurs annotations peuvent être empilées au-dessus d'une déclaration.

Exemples

```
@experimental
@version("0.2.0")
@scope(level=ScopeLevel.Internal)
@author(
  name="Greg",
  email="greg@example.com"
)
@deprecated(reason="Use new module instead")
@confidentiality(privacy.Level.HIGH)
module metadata.annotations
```

Règles

- Les valeurs peuvent être :
 - des chaînes : `"texte"`
 - des identifiants : `Internal`
 - des chemins qualifiés : `privacy.Level.HIGH`
- Les paires `clé=valeur` peuvent être combinées dans une annotation.
- Les annotations sont **optionnelles** et **non normatives** mais peuvent être exploitées par les outils CLI ou LSP.

[2025-09-10] – Annotations avec arguments typés

- **Contexte** : Besoin d'enrichir les artefacts avec des métadonnées structurées.
- **Décision** : Support des annotations avec arguments (valeur unique ou paires clé/valeur), et valeurs typées (`string`, `boolean`, `number` – y compris `hex` –, `qualifiedName`, `interval`).
- **Pourquoi** : Permet une expressivité maximale tout en restant compatible avec une grammaire ANTLR claire et extensible.

🔗 Déclaration des types d'annotations

Meshr permet de déclarer des **types d'annotations** personnalisés à l'aide du mot-clé `annotation`.

Syntaxe

```
@Target(annotation)
@Retention(model)
annotation Documented is
  required summary : String
  optional description : String = ""
  optional deprecated : Boolean = false
end
```

▣ Sémantique

- Les types d'annotations sont eux-mêmes annotables (`@Target` , `@Retention` , `@Repeatable` ...).
- Le bloc interne suit la forme :
 - `required` : la valeur doit obligatoirement être fournie à l'usage, **aucune valeur par défaut autorisée**.
 - `optional` : la valeur est optionnelle, et peut être associée à une valeur par défaut.
- Si une annotation typée est utilisée sans renseigner un champ `optional` , alors la valeur par défaut s'applique.
- Les types autorisés incluent les **types de base** (`String` , `Boolean` , `Integer` , `Float` , `Double` , `Date` , `Datetime` , `Time` , `Timestamp` , `Geography` , `Bytes` , `Json` , `Interval` , `Range`) et les **noms qualifiés** (enums, types importés).
- Le mot-clé `end` est requis pour clore la déclaration.

▣ Règle sémantique `required` vs `optional`

- `required` : champ **obligatoire, sans valeur par défaut**, doit être explicitement renseigné à l'usage.
- `optional` : champ **optionnel, avec ou sans valeur par défaut**.
 - Si valeur par défaut spécifiée → utilisée si champ non renseigné.
 - Sinon → champ absent à l'usage.

▣ Types de base autorisés pour les annotations

Les types suivants peuvent être utilisés dans les déclarations d'annotations :

- `Boolean` : valeurs `true` ou `false`
- `String` : chaînes de caractères délimitées par des guillemets
- `Integer` : nombre entier (ex: `42`)
- `Float` : nombre décimal (ex: `3.14`)
- `Double` : précision étendue (équivalent sémantique à `Float` pour l'instant)
- `Date` , `Datetime` , `Time` , `Timestamp` : types temporels
- `Geography` : localisation géographique (future extension)
- `Bytes` : données binaires (future extension)
- `Json` : valeurs encodées en JSON (future extension)
- `Interval` : intervalle sur un type temporel (ex: `Interval 2 hour`)
- `Range` : intervalle contigu entre deux valeurs ordonnées (ex: `Range 1..10`)

▣ Contraintes sur les types `String`

Les types `String` peuvent être enrichis avec des contraintes pour valider le format et la longueur des chaînes :

Syntaxe des contraintes

```
// Contrainte de pattern (regex)
email : String pattern "^[^@]+@[^@]+$"

// Contrainte de longueur
username : String length 3..20

// Contraintes combinées
phone : String pattern "[0-9]{10,15}" length 10..15
```

Types de contraintes

- **pattern** : expression régulière pour valider le format
 - Exemple email: `String pattern "^[^@]+@[^@]+$"`
 - Exemple téléphone: `String pattern "[0-9]{10,15}"`
 - Exemple URL: `String pattern "^https?://.*"`
- **length** : bornes sur la longueur de la chaîne
 - Longueur fixe: `String length 10..10`
 - Longueur variable: `String length 1..100`
 - Longueur minimale: `String length 5..`

Utilisation dans tous les contextes

Les contraintes `String` peuvent être utilisées partout où un type `String` est autorisé :

```
// Dans les annotations
annotation UserProfile is
    required email : String pattern "^[^@]+@[^@]+$"
    required username : String length 3..20
    optional phone : String pattern "[0-9]{10,15}$" length 10..15
end

// Dans les enums
enum HttpStatus(code: Integer, message: String pattern "[A-Z][a-z ]+$") is (
    ok(code = 200, message = "OK"),
    not_found(code = 404, message = "Not Found")
)

// Dans les records
record Contact is
    name : String length 1..100
    email : String pattern "^[^@]+@[^@]+$"
    phone : String pattern "[0-9]{10,15}$"
end

// Dans les traits
trait Identifiable is
    id : String pattern "[a-zA-Z0-9_-]+$" length 1..50
    name : String length 1..100
end
```

Règles sémantiques

- Les contraintes `pattern` et `length` peuvent être combinées
- L'ordre des contraintes n'est pas significatif
- Les contraintes sont optionnelles : `String` sans contrainte est valide
- Les erreurs de contraintes (double `pattern`, `length` incompatible) sont détectées par l'analyseur sémantique

Les annotations peuvent utiliser les valeurs suivantes:

- Chaînes: `"abc"`
- Booléens: `true`, `false`
- Numériques: `42`, `3.14`, `0xFF`, `0x00FF00`
- Noms qualifiés: `privacy.Level.HIGH`
- Intervalles: voir section dédiée ci-dessous

Les énumérations peuvent également être utilisées comme type d'attribut.

```
annotation Repeatable is
    optional value : Boolean = true
end

enum Color(value: Integer) is (
    red(value = 0xFF0000),
    green(value = 0x00FF00),
    blue(value = 0x0000FF)
)
```

Exemples

```
// Type d'annotation avec plusieurs champs
@Target(annotation)
@Retention(model)
annotation Example is
    required name : String
    optional version : String = "1.0"
    optional experimental : Boolean = false
end
```

```
// Annotation simple booléenne
@Target(annotation)
@Retention(model)
annotation Repeatable is
    optional value : Boolean = true
end
```

Tests valides

```
@Example(name="MeshrLang")
annotation Test1 is end

@Example(name="MeshrLang", experimental=true)
annotation Test2 is end
```

Tests invalides

(à venir, via fichier `invalid-annotation-decl-test.meshr`)

- Champ `required` non renseigné
- Redondance entre champs
- Mauvais type

Enums

Les **énumérations** permettent de représenter un ensemble fini de valeurs symboliques, typiquement utilisées pour encoder des catégories, des statuts ou des niveaux de sensibilité.

Syntaxe simple (inline)

```
export enum PIICategory is (contact, identity, financial, health, biometric, location)
```

- Le mot-clé `enum` introduit une énumération nommée.
- L'utilisation de `export` rend l'énumération visible à l'extérieur du module.
- Les valeurs sont listées entre parenthèses, séparées par des virgules.
- Cette forme convient aux déclarations rapides et lisibles.

Export groupé

```
export { PIICategory }

enum PIICategory is (contact, identity, financial, health, biometric, location)
```

- L'énumération est déclarée sans `export`, puis rendue publique via un bloc `export { ... }`.
- Cette forme permet de centraliser tous les artefacts publics après les imports.

Règles

- Les valeurs d'énum peuvent être des **identifiants** ou des **chaînes** (ex: `"export"`).
- Les identifiants de valeurs doivent être uniques et respectent la casse.
- Une énumération peut être annotée, comme tout autre artefact.
- Le nom de l'énumération doit être un identifiant valide.

[2025-09-10] – Enums avec attributs typés

- **Contexte** : Il est utile de donner aux énumérations des attributs associés à chaque valeur (ex. code couleur, libellé, gravité).
- **Décision** : On introduit une forme enrichie de déclaration d' `enum`, où chaque valeur peut porter une ou plusieurs **propriétés typées**, similaires aux `enum class` de Kotlin.
- **Pourquoi** : Cela augmente l'expressivité du langage, permet des relations croisées entre enums, et reste compatible avec les aspects et politiques du langage.

Syntaxe enrichie

```

enum Color(value: Integer) is (
  red(value = 0xFF0000),
  green(value = 0x00FF00),
  blue(value = 0x0000FF)
)

enum Severity(color: Color, severity: String = "none") is (
  none(color = Color.blue),
  medium(color = Color.green, severity = "medium"),
  high(color = Color.red, severity = "high")
)

```

❗ Remarque : les valeurs entières peuvent être exprimées en hexadécimal (ex: 0xFF0000) pour représenter des couleurs ou des flags binaires.

❗ Règles

- Une énumération peut définir une **signature d'attributs** dans (...) immédiatement après le nom.
- Chaque valeur de l'énumération doit fournir des arguments nommés (ex: color = Color.red).
- Les attributs peuvent avoir une **valeur par défaut**, comme severity: String = "none" .
- Les types d'attributs peuvent être des **types de base** (cf. liste) ou des **noms qualifiés** (ex.: autre enum).
- L'ordre des arguments dans les valeurs n'a pas besoin de suivre celui de la signature.

❗ Les nombres peuvent être fournis en **hexadécimal** (0xFF0000) pour des codes couleur, et les **intervalles** sont supportés dans les valeurs d'annotation.

❗ Intervalles (Interval)

Les littéraux d'intervalle sont supportés dans les annotations via la forme `Interval ...` .

Formes supportées:

- Partie simple: `Interval 12 month` , `Interval -1 day` , `Interval 2 hour`
- Avec bornes textuelles + précision: `Interval "2024-01" year to month`
- Précisions supportées: `year` , `quarter` , `month` , `week` , `day` , `hour` , `minute` , `second` , `millisecond` , `microsecond`

Notes:

- Le signe - est autorisé devant les nombres d'intervalle: `Interval -10 day` .
- Les nombres hexadécimaux sont acceptés: `Interval 0x10 second` .

❗ Types composites: List, Map, Range, Record

Meshr supporte des types composites utilisables dans les signatures d'énumérations et les déclarations d'annotations via `typeRef` :

- `List of T`
- `Map of K to V`
- `Range of T`
- `record` référencé par nom (voir ci-dessous)

Exemples de types:

```

annotation Meta is
  required owner : String
  optional tags : List of String
  optional conf : Map of String to Integer
  optional span : Range of Integer
end

record Address is
  street : String
  zipcode : Integer
  extras : Map of String to Integer
end

```

Valeurs par défaut dans record

Les champs d'un `record` peuvent recevoir une valeur par défaut:

```
record Address is
  street : String = "foo"
  zipcode : Integer = 42
  extras : Map of String to Integer = Map { "foo" : 42, "bar" : 69 }
end
```

Littéraux composites utilisables dans `annotationValue` et les valeurs d'`enum`

- List: `List[1,2,3]`, `List["a","b"]`
- Map: `Map{"k1":1, "k2":2}`
- Record anonyme: `Record{name:"Alice", age:42}`
- Range: `Range -2048 .. 2048`, `Range 1..10`

Exemple d'usage dans une `enum`:

```
enum Product(status: String, owners: List of String, address: Address, limits: Range of Integer) is (
  active(status = "ok", owners = List["ops","qa"], address = Record{street:"A", zipcode:75001, extras:Map{"r":1}}, limits = Range 1.
)
```

Exemples avancés (imbriqués)

```
record Contact is
  name      : String
  emails    : List of String
  attributes: Map of String to String = Map{"role":"owner"}
end

record Team is
  name      : String
  members   : List of Contact
  quotas    : Map of String to Range of Integer = Map{"daily": Range -100 .. 100}
end

annotation Project is
  required name      : String
  optional team      : Team
  optional labels    : List of String = List["alpha","beta"]
  optional routing   : Map of String to Map of String to Integer
end

@Project(
  name = "mesh",
  team = Record{
    name: "core",
    members: List[
      Record{name:"Alice", emails: List["a@example.com"]},
      Record{name:"Bob",   emails: List["b@example.com","bob@work"]}
    ],
    quotas: Map{"daily": Range -50 .. 50}
  },
  routing = Map{
    "svcA": Map{"p": 80, "s": 20},
    "svcB": Map{"p": 60, "s": 40}
  }
)

module demo.advanced
```

⚡ Traits

Un trait est un bloc réutilisable de déclarations (champs, métadonnées, contraintes, aspects, politiques, ...) injectables dans d'autres artefacts via la clause `with`. Les traits favorisent la factorisation et la composition déclarative, sans comportement impératif.

⚡ Syntaxe


```
trait <Identifiant> [with Base1, Base2] is
  <FieldName> : typeRef [= <valeur>]
  ...
end
```

🔗 Injection via `with`

La clause `with` permet d'injecter un ou plusieurs traits dans un artefact. Aujourd'hui, elle est supportée sur `record` et `trait`.

```
trait Audit is
  CreatedAt : Timestamp
  UpdatedAt : Timestamp
end

record Contact with Audit is
  Id : String
end

trait Contact is
  Email : String
end

trait ExtendedContact with Contact is
  Phone : String
end

record Person with Contact, Audit is
  Name : String
end
```

🔗 Règles de composition (sémantique)

- Monotonie: on ne relâche pas les exigences (required → optional interdit, suppression de contraintes interdite).
- Intersection: des contraintes compatibles s'additionnent; on garde la version la plus restrictive.
- Explicitation: en cas de conflit dur (types différents, contraintes incompatibles, aspects contradictoires), l'artefact consommateur doit redéfinir explicitement.

Récapitulatif:

Situation	Exemple	Résolution
Compatibilité parfaite	X: string + X: string	Fusion automatique
Extension compatible	Y: string + Y: string required	required l'emporte
Contraintes compatibles	Z: string + Z: string pattern "[A-Z]+\$"	Conserve la contrainte (intersection)
Conflit (types)	A: int + A: string	Erreur → redéfinir dans l'artefact
Conflit (contraintes)	B: string length 1..10 + B: string length 20..30	Erreur → redéfinir
Conflit (aspects)	C aspects { PII{level:high} } + C aspects { PII{level:low} }	Erreur → redéfinir

Notes:

- Ces règles sont vérifiées par les outils (validation sémantique), pas à l'analyse syntaxique.
- `typeRef` est autorisé dans les champs de `trait`, de `record`, dans les attributs d'énum et dans les champs d'annotations.

🔗 [2025-09-09] — Module = unité, namespace et fichier unique

- **Contexte** : Il fallait décider si l'on autorisait plusieurs modules par fichier, et s'il y avait un lien fort avec l'arborescence.
- **Décision** : Le langage impose qu'un fichier `.meshr` déclare **un seul module**. Le nom du module est un **namespace**. L'arborescence n'est **pas contrainte**, mais une convention est proposée.
- **Pourquoi** : Cela permet une résolution simple par les outils (CLI, LSP), évite les collisions de symboles, et reste lisible.

☒ Artefacts définissables

1. domain

Déclaration d'un domaine hiérarchique.

```
domain retail.marketing {
  description = "Marketing domain for the retail perimeter"
}
```

2. product

Déclaration d'un produit de données.

```
product leads_b2c_import {
  domain = retail.marketing
  contracts = [
    contract:marketing:leads_b2c_import:1.0.0
  ]
}
```

3. contract

(Défini dans une autre couche ou généré via DCDL)

4. aspect

(TBD)

5. team

(TBD)

6. policy

(TBD)

☒ Décisions de conception

☒ [2025-09-09] — Syntaxe déclarative avec blocs `{ }`

- **Contexte** : Le langage doit rester simple, lisible pour les métiers, mais formalisable en grammaire ANTLR.
- **Décision** : Utiliser une syntaxe à blocs (`{ }`), proche de HCL/Terraform.
- **Pourquoi** : Meilleure lisibilité et extensibilité que YAML/JSON, tout en étant facilement parsable.

☒ [2025-09-10] — Visibilité par export explicite

- **Contexte** : Le langage devait gérer la visibilité entre modules pour encourager l'encapsulation.
- **Décision** : Un artefact déclaré dans un module n'est accessible depuis l'extérieur que s'il est précédé du mot-clé `export` .
- **Pourquoi** : Cette règle encourage une séparation claire entre API publique et éléments internes, facilite la documentation automatique et le linting.

☒ [2025-09-10] — Deux formes d'export : inline ou groupé

- **Contexte** : Besoin de contrôler la visibilité des artefacts à l'extérieur du module.
- **Décision** : Deux formes sont supportées : inline (`export decl`) et groupée (`export { A, B }`).
- **Justification** : Favorise à la fois la lisibilité locale (inline) et la gestion explicite de l'API publique (groupé après les imports).

☒ À venir

- Structuration des aspects (inline vs référence)
- Système de types
- Imports / Références croisées
- Mécanisme de validation

Annexes

Annexe A — Grammaire EBNF (référence)

```
(* =====
  Meshr-Lang — Grammaire EBNF (alignée ANTLR)
  Mise à jour: 2025-09-10
  ===== *)

compilation-unit      = module-decl, { import-decl }, { export-decl }, { top-level-decl }, EOF ;

module-decl           = { annotation }, "module", qualified-name ;

import-decl           = "import", import-items, "from", qualified-name ;
import-items          = "*"
                      | identifier
                      | "{", import-item, { ",", import-item }, ")" ;
import-item           = identifier ;

export-decl           = "export", ( export-items | exportable-decl ) ;
export-items          = "{", identifier, { ",", identifier }, ")" ;

top-level-decl        = enum-decl | annotation-decl ;
exportable-decl       = enum-decl ;

enum-decl             = { annotation }, "enum", identifier, [ enum-signature ], "is", "(", enum-values, ")" ;
enum-signature        = "(", enum-attribute, { ",", enum-attribute }, ")" ;
enum-attribute        = identifier, ":", ( qualified-name | base-type ), [ "=", annotation-value ] ;
enum-values           = enum-value, { ",", enum-value } ;
enum-value            = (identifier | string-literal), [ "(", enum-value-args, ")" ] ;
enum-value-args       = enum-value-arg, { ",", enum-value-arg } ;
enum-value-arg        = identifier, "=", annotation-value ;

annotation-decl       = { annotation }, "annotation", identifier, "is", annotation-field, { annotation-field }, "end" ;
annotation-field      = ( "required" | "optional" ), identifier, ":", ( qualified-name | base-type ), [ "=", annotation-value ], [ NEW

annotation            = "@", identifier, [ "(", [ annotation-args ], ")" ] ;
annotation-args       = annotation-arg, { ",", annotation-arg } ;
annotation-arg        = annotation-arg-pair | annotation-value ;
annotation-arg-pair   = identifier, "=", annotation-value ;
annotation-value      = string-literal | number-literal | boolean-literal | qualified-name | interval-literal ;

base-type             = "Boolean" | string-type | "Integer" | "Float" | "Double"
                      | "Date" | "Datetime" | "Time" | "Timestamp"
                      | "Geography" | "Bytes" | "Json"
                      | "Interval" | "Range" ;

(* Type String avec contraintes optionnelles *)
string-type           = "String", [ string-constraints ] ;
string-constraints    = string-constraint, { string-constraint } ;
string-constraint     = "pattern", string-literal
                      | "length", signed-number, "..", signed-number ;

interval-literal      = "Interval", interval-single-part
                      | "Interval", string-literal, "year", "to", "month"
                      | "Interval", string-literal, "year", "to", "day"
                      | "Interval", string-literal, "year", "to", "hour"
                      | "Interval", string-literal, "year", "to", "minute"
                      | "Interval", string-literal, "year", "to", "second"
                      | "Interval", string-literal, "month", "to", "day"
                      | "Interval", string-literal, "month", "to", "hour"
```

```

| "Interval", string-literal, "month", "to", "minute"
| "Interval", string-literal, "month", "to", "second"
| "Interval", string-literal, "day", "to", "hour"
| "Interval", string-literal, "day", "to", "minute"
| "Interval", string-literal, "day", "to", "second"
| "Interval", string-literal, "hour", "to", "minute"
| "Interval", string-literal, "hour", "to", "second"
| "Interval", string-literal, "minute", "to", "second" ;

interval-single-part = [ '-' ], number-literal, interval-unit ;
interval-unit
    = "year" | "quarter" | "month" | "week" | "day"
    | "hour" | "minute" | "second" | "millisecond" | "microsecond" ;

qualified-name
    = identifier, { ".", identifier } ;
boolean-literal
    = "true" | "false" ;

string-literal
    = "'", { character - "'" | '\\' }, "'" ;
number-literal
    = digit, { digit } | "0x", hex-digit, { hex-digit } ;
identifier
    = letter, { letter | digit | "_" } ;
NEWLINE
    = ("\r"?), "\n", { ("\r"?), "\n" } ;

```

Annexe B — Grammaire ANTLR (référence, Python target)

```

grammar MeshrModule;

// =====
// Entrée principale
// =====
compilationUnit
    : annotatedModuleDecl importDecl* exportDecl* topLevelDecl* EOF
    ;

// =====
// Déclarations de module
// =====
annotatedModuleDecl
    : annotation* 'module' qualifiedName
    ;

// =====
// Import / Export
// =====
importDecl
    : 'import' importItemsWithOptionalBraces 'from' qualifiedName
    ;

importItemsWithOptionalBraces
    : IDENTIFIER                # SingleImport
    | '*'                       # WildcardImport
    | '{' importItems '}'       # GroupImport
    ;

importItems
    : IDENTIFIER (',' IDENTIFIER)+
    ;

exportDecl
    : 'export' exportableDecl    # InlineExport
    | 'export' '{' exportItems '}' # GroupedExport
    ;

exportItems
    : IDENTIFIER (',' IDENTIFIER)*
    ;

```

```

,

// =====
// Déclarations de haut niveau
// =====
topLevelDecl
  : annotatedEnumDecl
  | annotatedAnnotationDecl
  | traitDecl
  | recordDecl
  ;

// ===== ENUM =====
exportableDecl
  : enumDecl
  ;

annotatedEnumDecl
  : annotation* enumDecl
  ;

enumDecl
  : 'enum' IDENTIFIER enumSignature? 'is' '(' enumValueList ')'
  ;

enumSignature
  : '(' enumAttributeList ')'
  ;

enumAttributeList
  : enumAttribute (',' enumAttribute)*
  ;

enumAttribute
  : IDENTIFIER ':' typeRef ('=' annotationValue)?
  ;

enumValueList
  : enumValue (',' enumValue)*
  ;

enumValue
  : (IDENTIFIER | STRING_LITERAL) ('(' enumValueArgList? ')')?
  ;

enumValueArgList
  : enumValueArg (',' enumValueArg)*
  ;

enumValueArg
  : IDENTIFIER '=' annotationValue
  ;

// ===== ANNOTATION DECLARATION =====
annotatedAnnotationDecl
  : annotation* annotationDecl
  ;

annotationDecl
  : 'annotation' IDENTIFIER 'is' annotationFieldList 'end'
  ;

annotationFieldList
  : annotationField+
  ;

```

```

annotationField
    : ('required' | 'optional') IDENTIFIER ':' typeRef ('=' annotationValue)? NEWLINE?
    ;

// ===== ANNOTATION USAGE =====
annotation
    : '@' IDENTIFIER '('(' annotationArgs? ')')?
    ;

annotationArgs
    : annotationArg (',' annotationArg)*
    ;

annotationArg
    : annotationArgPair
    | annotationValue
    ;

annotationArgPair
    : IDENTIFIER '=' annotationValue
    ;

// ===== TYPES DE BASE =====

// Types de base reconnus
baseType
    : 'Boolean'
    | stringType
    | 'Integer'
    | 'Float'
    | 'Double'
    | 'Date'
    | 'Datetime'
    | 'Time'
    | 'Geography'
    | 'Timestamp'
    | 'Bytes'
    | 'Json'
    | 'Interval'
    | 'Range'
    ;

// Type String avec contraintes optionnelles
stringType
    : 'String' stringConstraints?
    ;

stringConstraints
    : stringConstraint (stringConstraint)*
    ;

stringConstraint
    : 'pattern' STRING_LITERAL
    | 'length' signedNumber '..' signedNumber
    ;

// ===== SHARED =====
qualifiedName
    : IDENTIFIER ('.' IDENTIFIER)*
    ;

// ===== TYPE REFERENCES =====
typeRef
    : qualifiedName
    | baseType
    ;

```

```

| listType
| mapType
| rangeType
;

listType
: 'List' 'of' typeRef
;

mapType
: 'Map' 'of' typeRef 'to' typeRef
;

rangeType
: 'Range' 'of' typeRef
;

// ===== ANNOTATION USAGE =====
annotationValue
: STRING_LITERAL
| NUMBER_LITERAL
| BOOLEAN_LITERAL
| qualifiedName
| intervalLiteral
| listLiteral
| mapLiteral
| recordLiteral
| rangeLiteral
;

// ===== INTERVAL LITERALS =====
intervalLiteral
: 'Interval' intervalSinglePart
| 'Interval' STRING_LITERAL 'year' 'to' 'month'
| 'Interval' STRING_LITERAL 'year' 'to' 'day'
| 'Interval' STRING_LITERAL 'year' 'to' 'hour'
| 'Interval' STRING_LITERAL 'year' 'to' 'minute'
| 'Interval' STRING_LITERAL 'year' 'to' 'second'
| 'Interval' STRING_LITERAL 'month' 'to' 'day'
| 'Interval' STRING_LITERAL 'month' 'to' 'hour'
| 'Interval' STRING_LITERAL 'month' 'to' 'minute'
| 'Interval' STRING_LITERAL 'month' 'to' 'second'
| 'Interval' STRING_LITERAL 'day' 'to' 'hour'
| 'Interval' STRING_LITERAL 'day' 'to' 'minute'
| 'Interval' STRING_LITERAL 'day' 'to' 'second'
| 'Interval' STRING_LITERAL 'hour' 'to' 'minute'
| 'Interval' STRING_LITERAL 'hour' 'to' 'second'
| 'Interval' STRING_LITERAL 'minute' 'to' 'second'
;

intervalSinglePart
: '-'? NUMBER_LITERAL intervalUnit
;

intervalUnit
: 'year' | 'quarter' | 'month' | 'week' | 'day'
| 'hour' | 'minute' | 'second' | 'millisecond' | 'microsecond'
;

// ===== RECORD DECLARATION =====
recordDecl
: 'record' IDENTIFIER withClause? 'is' recordFieldList 'end'
;

recordFieldList
: recordField+

```

```

;

recordField
  : IDENTIFIER ':' typeRef ('=' annotationValue)? NEWLINE?
  ;

// ===== TRAIT DECLARATION =====
traitDecl
  : 'trait' IDENTIFIER withClause? 'is' traitFieldList 'end'
  ;

traitFieldList
  : traitField+
  ;

traitField
  : IDENTIFIER ':' typeRef ('=' annotationValue)? NEWLINE?
  ;

withClause
  : 'with' qualifiedName (',' qualifiedName)*
  ;

// ===== COLLECTION AND COMPOSITE LITERALS =====
listLiteral
  : 'List' '[' (annotationValue (',' annotationValue)*)? ']'
  ;

mapLiteral
  : 'Map' '{' (mapEntry (',' mapEntry)*)? '}'
  ;

mapEntry
  : annotationValue ':' annotationValue
  ;

recordLiteral
  : 'Record' '{' (recordLitField (',' recordLitField)*)? '}'
  ;

recordLitField
  : IDENTIFIER ':' annotationValue
  ;

rangeLiteral
  : 'Range' signedNumber '..' signedNumber
  ;

signedNumber
  : '-'? NUMBER_LITERAL
  ;

fragment ESC
  : '\\' ["\\/bfnrt]
  ;

AT: '@';
STRING_LITERAL: '"' (ESC | ~["\\r\n])* '"';
IDENTIFIER: [a-zA-Z_][a-zA-Z0-9_]* ;

WS: [ \t\r\n]+ -> skip ;
NEWLINE: ('\r'? '\n')+ -> skip ;
COMMENT: '//' ~[\r\n]* -> skip ;
MULTILINE_COMMENT: '/*' .*? '*/' -> skip ;

```


NUMBER_LITERAL

```
: [0-9]+ ('.' [0-9]+)?  
| '0' [xX] [0-9a-fA-F]+  
;
```

BOOLEAN_LITERAL

```
: 'true' | 'false'  
;
```